Fitted — Project 2 Proposal

Oct 11, 2026 · Group 1

Group 1: Marianna Belmares · Alexandra Gutierrez Aguilar · Brent Paterson · Nico Savaides

App purpose

Fitted becomes an AI stylist: it learns a user's taste from their Pinterest boards or Google Photos, listens to what they need in their own voice, and builds complete outfits they can see on themselves before buying.

Online shoppers already collect what they like on Pinterest, and keep outfit photos in Google Photos, but that inspiration never turns into a plan for what to wear. In Project 2, a user signs in, connects Pinterest or Google Photos, and records a short voice request such as *"I need an outfit for a summer wedding, nothing too formal, under $200."* Fitted then:

1. **Reads their style.** Gemini analyzes the outfit images on the Pinterest boards or Google Photos they choose and builds a *Style DNA* profile: colors, silhouettes, fabrics and aesthetic.
2. **Understands the request.** Gemini turns the voice recording into a structured *style brief*: occasion, weather, formality, budget and must-avoid items.
3. **Builds looks.** A team of Google ADK agents combines the Style DNA and the brief to choose pieces from the Fitted catalog, explain why each one works, and generate a virtual try-on preview on the user's own photo.
4. **Remembers.** Profiles, briefs and saved looks are stored in Firestore, so **My Looks** and future recommendations improve each time.

Output the user sees: 2–3 complete *looks* per request. Each shows the pieces with prices, a short stylist note linking it to their pins and their words, a try-on preview, and buttons to save or ask for another option.

Who it's for: people who shop online for clothes, save outfit ideas on Pinterest or in their photos, and want help turning that inspiration into something they can actually wear.

Authentication

Users sign in to Fitted with Google, then connect **one photo source: Pinterest (Option A) or Google Photos (Option B).** API keys, client secrets and tokens never reach the browser.

Fitted account (both options): Google Sign-In through Firebase Authentication.

- The Project 1 Login page keeps its layout but gains a **Continue with Google** button.
- Firebase returns an ID token to React.
- React sends it as `Authorization: Bearer <token>` on every `/api/*` call, and an Express `requireAuth` middleware verifies it with `firebase-admin` before any route runs.
- The verified `uid` is the key for all of that user's Firestore data.
- Requiring sign-in also protects our Gemini API key, which lives only on the server (Secret Manager).

Option A — Pinterest (OAuth 2.0, Pinterest API v5)

1. On the **Connect** screen, the user clicks **Connect Pinterest**. React calls `GET /api/photos/pinterest/auth`.
2. Express redirects to Pinterest's authorization page with our `client_id`, `redirect_uri`, `response_type=code`, a random `state` value, and the scopes `user_accounts:read`, `boards:read` and `pins:read`.
3. The user logs in to *their own* Pinterest account and approves.
4. Pinterest redirects to `/api/photos/pinterest/callback`. Express checks `state`, then exchanges the code at `https://api.pinterest.com/v5/oauth/token` using our client secret (HTTP Basic auth, server only).
5. Express stores the access token (valid 30 days) and refresh token (valid 60 days) in the user's private Firestore document and refreshes them when needed.
6. The user ticks the boards that represent their style. **Adding a new image** means saving a new pin to one of those boards.

Requirements and risk: the app needs a Pinterest **business** account (our Project 1 site on AWS is the business website), a verified email, and acceptance of the Developer Terms. Pinterest reviews app requests each business day, and new apps start on *Trial* access (1,000 calls/day).

Since September 14, 2026, new apps need extra approval to read *other* people's boards and pins. Apps can still read boards and pins owned by the account that authorized them, which is all Fitted needs. Pinterest's docs don't say whether a Trial app can be authorized by accounts other than the developer's, and our professor must be able to log in with his own account. We will test this with two team accounts before relying on Option A.

Option B — Google Photos (Google Photos Picker API)

1. On the **Connect** screen, the user clicks **Choose from Google Photos**. Google asks once for the read-only picker permission (`photospicker.mediaitems.readonly`), using the same Google account they signed in with.
2. Express creates a picker session (`POST https://photospicker.googleapis.com/v1/sessions`) and returns its `pickerUri` . React opens it in a new window with `/autoclose` appended.
3. The user selects outfit photos in Google Photos: screenshots of looks they like, or photos of themselves in clothes they love.
4. Express polls the session until `mediaItemsSet` is true, lists the picked items, and downloads each image from its `baseUrl` with the user's access token. Fitted keeps its own copy in Cloud Storage, because these links expire.
5. **Adding a new image** means taking or saving a new photo in Google Photos and picking it on the Connect screen.

Requirements and risk: only a Google Cloud OAuth consent screen and the Picker API enabled in the project we already use. No business account or partner review is needed. Google limits apps to the photos the user explicitly picks (broad library access ended April 1, 2025), which is how Fitted is meant to work anyway. While the app is in testing mode, each user, including our professor, is added as a test user on the consent screen.

Comparison and decision

	Option A: Pinterest	Option B: Google Photos
What the photos mean	Inspiration the user has collected on boards	Photos the user chooses: inspiration screenshots or their own outfits
Login	Separate Pinterest OAuth	Same Google account as Fitted sign-in
Setup	Business account + app review	Consent screen + enable API

	Option A: Pinterest	Option B: Google Photos
Can the professor use it?	Not confirmed for Trial apps	Yes, once added as a test user
Fit for a fashion app	Strongest: boards are already curated	Good: user picks what represents their style
Main risk	Approval and access limits	Picked-photo links expire (we copy images)

Decision: the backend has one `photoSourceService` with two adapters, `pinterestAdapter` and `googlePhotosAdapter`, that return the same image list. Everything after that (Gemini, agents, Firestore, React screens) is identical. We are registering the Pinterest app and enabling the Picker API this week. If Pinterest access for outside accounts is confirmed by **October 20**, we ship both and the user chooses on the Connect screen; otherwise Google Photos is the primary source.

How Gemini processes photos and audio

Fitted calls the **Gemini API** (not Vertex AI) from Express through the `@google/genai` Node SDK, using an API key stored in Secret Manager. Every understanding call returns **structured JSON** that matches a schema we define, so results can be saved to Firestore and drawn in React without fragile text parsing. Model names below are the ones in Google's current docs; we'll pin exact versions when we build.

#	Call	Input	Model	Output (JSON unless noted)
P1	Pin analysis	Up to ~10 outfit images per request, from the user's chosen Pinterest boards (A) or picked Google Photos (B)	Gemini Flash (`gemini-3.8-flash`)	Per image: garments with type, color name and hex, silhouette, fabric, pattern, formality (1–5), season, and aesthetic tags (e.g. "minimal", "coastal", "90s")

#	Call	Input	Model	Output (JSON unless noted)
P2	Photo check	The user's own full-body photo (uploaded, as in Project 1)	Gemini Flash	Is the photo usable for try-on (full body, front-facing, lighting, one person)? If not, a short reason and a tip to retake it
P3	Virtual try-on	User photo + 1–3 product images from the chosen look	Nano Banana 2.1 (`gemini-nano-banana-2.1`), which accepts up to 14 reference images	A generated image of the user wearing the look, saved to Cloud Storage
A1	Voice style request	10–60 s recording from the browser microphone (WebM/Opus in Chrome and Edge, MP4/AAC in Safari, sent inline)	Gemini Flash	Transcript plus a *style brief*: occasion, date/season, setting and weather, formality, budget, colors wanted/avoided, items wanted/avoided, and mood
A2	Voice refinement	A short follow-up such as *"less formal, and no heels"*, plus the current look's JSON	Gemini Flash	A list of changes to the brief, which the agents use to rebuild the look

Why this is non-trivial:

- **Photos turn into style data.** P1 doesn't label images. It extracts garment-level attributes from many pins or photos, which the agents combine into a long-lived *Style DNA* profile.
- **Audio carries what pins can't.** Pins say what someone likes; the voice note says what they need *right now*: occasion, budget and limits. A1 turns natural speech into the same structured fields the agents filter on.

- **Photo and audio results are combined.** Every look is chosen by matching the voice brief against the Style DNA and the catalog. The stylist note cites both (*"you pin a lot of linen and asked for something breathable"*).
- **Photos are also generated.** P3 produces the try-on preview that Project 1 left as a placeholder.

Limits we design around: inline requests are capped at 20 MB, so recordings are limited to 60 seconds. Audio costs about 32 tokens per second (a 30-second note is roughly 1,000 tokens). Inspiration images are resized on the server before they are sent. Try-on (P3) only works with real clothing photos, so the catalog uses real product images (see Product catalog).

Google ADK agent architecture

A four-agent **Stylist pipeline**, built with ADK for TypeScript (`@google/adk`), turns the Gemini photo and audio results into finished, checked looks. It is **deployed as its own Cloud Run service** (`fitted-agents`), separate from the Express API, so the agents can be deployed, scaled and demonstrated on their own.

The pipeline uses ADK's workflow agents: a **sequential** workflow with a **loop** in the middle.

`StyleAnalyst` → loop(`LookBuilder` ⇄ `LookCritic` , max 2 rounds) → `Presenter`

Agent	Type	Tools it can call	What it does
StyleAnalyst	LlmAgent	`getPhotoAnalyses(uid)` , `analyzeNewPhotos(uid)` (Gemini P1), `getFeedbackHistory(uid)`	Merges per-photo results and past feedback into the user's **Style DNA**: top colors, silhouettes, fabrics, aesthetics, and what they avoid. Re-analyzes only photos added since the last run.

Agent	Type	Tools it can call	What it does
LookBuilder	LlmAgent	`searchCatalog(filters)`, `getProduct(id)`	Uses the style brief (from audio) and Style DNA (from photos) to build 3 complete looks from the catalog: top/dress, bottom, outerwear when needed.
LookCritic	LlmAgent	`checkBudget(look, budget)`, `checkAvoidList(look, brief)`	Scores each look against the brief: budget, formality, weather, and items to avoid. Sends failing looks back to LookBuilder with reasons, for up to 2 rounds.
Presenter	LlmAgent	`renderTryOn(photo, products)` (Gemini P3), `saveLook(uid, look)`	Writes the stylist note for each look, which cites the inspiration photos and the voice request. It renders a try-on for the top look and saves everything to Firestore.

Meaningful use in the SPA:

- The agents produce the **Results** screen. Each look card is one Presenter output.
- The **critic loop** is visible: each card shows a small *"checked against your brief"* badge listing budget, weather and avoided items.
- **Voice refinement** (Gemini A2) re-runs only LookBuilder → LookCritic → Presenter in the same ADK session, so the agents remember the looks they already proposed.
- Agent steps stream to React as progress messages (*"Reading your boards…", "Building looks…", "Checking budget…"*), replacing Project 1's simulated *"Preparing your look…"* state.

Deploying the agents: the pipeline runs in a small Node server that wraps the ADK `Runner`, built into its own container and deployed with `gcloud run deploy fitted-agents` (ADK supports Cloud Run, GKE and Google's managed Agent Runtime). The service is private: Cloud Run only accepts calls carrying an identity token from the Express service's service account. Express starts a run, forwards the agents' progress events to React over Server-Sent Events, and returns the final looks. ADK sessions keep each user's run state, so a voice refinement continues the same conversation. The wiki's *ADK use and deployment* screenshots will show this service in the Cloud Run console and its agent logs.

What is stored in Firestore

Firestore (Native mode) stores everything keyed by the user's Firebase `uid`, so each person sees only their own profile, briefs and looks. Large binary files (the user photo, try-on images, voice recordings) go to Cloud Storage, and Firestore keeps their paths.

Collection path	One document holds	Written by	Read by
`users/{uid}`	Name, email, Cloud Storage path of the user's photo, `photoSource` (`pinterest` or `googlePhotos`), chosen Pinterest board IDs, and **Style DNA** (top colors, silhouettes, fabrics, aesthetics, avoid list, last updated)	Sign-in, Connect screen, StyleAnalyst	Every screen

Collection path	One document holds	Written by	Read by
`users/{uid}/private/tokens`	OAuth tokens for the connected source (Pinterest access/refresh tokens or Google Photos access token) with expiry times. Security rules block all browser access; only Express (Admin SDK) reads it.	OAuth callbacks, token refresh	Express only
`users/{uid}/photoAnalyses/{photoId}`	Source (`pinterest` or `googlePhotos`), source item ID (pin or media item), board ID if any, Cloud Storage copy of the image, Gemini P1 result (garments, colors, formality, tags), analyzed-at time	Gemini P1 via StyleAnalyst	StyleAnalyst, **My Style** screen
`users/{uid}/briefs/{briefId}`	Transcript, structured brief (occasion, weather, formality, budget, wants and avoids), recording length, audio MIME type, audio file path, created-at time	Gemini A1 / A2	LookBuilder, LookCritic, **History**
`users/{uid}/looks/{lookId}`	Brief ID, product IDs, total price, stylist note, critic results, try-on image path, saved flag, thumbs up/down, created-at time	Presenter; user actions	**Results** and **My Looks** screens

Collection path	One document holds	Written by	Read by
`products/{productId}`	Catalog items: name, category, price, colors, formality, season, tags, Cloud Storage path of the real product photo, image credit	Seed script	LookBuilder (`searchCatalog`), **Shop**
`events/{eventId}`	Server log for the Analytics report: route, model, latency in ms, token counts, success or error, `uid`	Express logging middleware	Analytics report

Main queries from the app:

- **My Looks:** `looks` where `saved == true`, ordered by `createdAt` descending.
- **Re-analysis:** `photoAnalyses` where `analyzedAt` is newer than the last Style DNA update, so only new pins or picked photos are sent to Gemini.
- **Catalog search:** `products` where `category` is in the needed categories and `price <= budget` (composite index), then ranked by the agent.
- **Updates and removals:** saving or unsaving a look updates `saved`; deleting a look removes its document; disconnecting a photo source deletes `private/tokens` and that user's `photoAnalyses`.

Product catalog

The catalog grows from Project 1's 10 SVG placeholders to about **48 real products** (12 each in Tops, Outerwear, Dresses and Bottoms), because virtual try-on needs real clothing photos and the agents need enough choice to build three different looks.

- **Images:** real product photos on plain backgrounds, either clothing team members own and photograph themselves, or images from free-license stock sites (credited in each product's `imageCredit`). They are stored in Cloud Storage, not hot-linked.
- **Data:** each product has name, category, price, colors, formality (1–5), season and style tags, so `searchCatalog` can filter by the brief.

- **Seeding:** a `seedProducts.js` script, which replaces Project 1's `products.js`, writes the items to Firestore.
- **Later:** the same `products` shape can be filled from a real store API (for example a Shopify development store) without changing the agents or React.

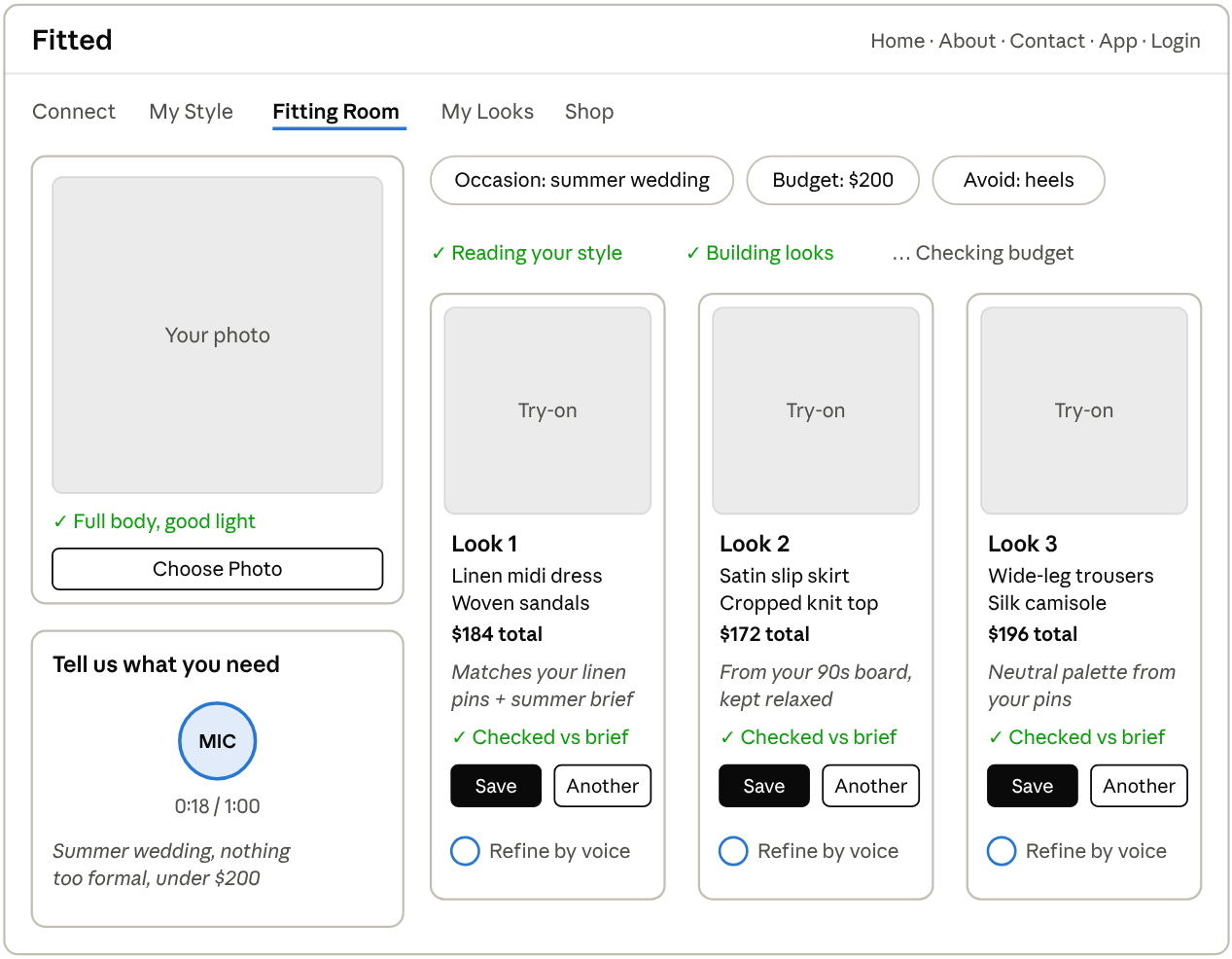
Mockup interfaces

The app keeps Project 1's look (warm neutrals, serif headings, thin borders) and its tabs, adding **Connect** and **My Style** and turning the Fitting Room into the main voice-to-looks flow. Three wireframes below cover the Fitting Room with results, the Connect screen for both photo options, and My Style with My Looks. We will also rebuild them in Figma for the submitted PDF.

SPA tabs: Connect · My Style · Fitting Room · My Looks · Shop

1. **Login** (Project 1 page). Same two-column layout and React state logic, plus a **Continue with Google** button above the username and password fields.
2. **Connect** (new). Two cards side by side: *Pinterest* (**Connect Pinterest**, then a grid of the user's boards with checkboxes) and *Google Photos* (**Choose from Google Photos**, then thumbnails of the picked photos with **Add more**). **Analyze my style** runs Gemini on the selection. If only one option ships, only its card appears.
3. **My Style** (new). The Style DNA: color swatches with names, aesthetic tags (*minimal · coastal · tailored*), top silhouettes and fabrics, and an avoid list. Below it, the analyzed pins or photos, each with small Gemini tags. **Refresh** re-analyzes new ones.
4. **Fitting Room** (evolved). On the left, Project 1's photo uploader with a Gemini photo-check message (*"Great — full body, good light"*). On the right, a **Tell us what you need** card: a large mic button with a 60-second timer, then the transcript and the parsed brief as editable chips (*Occasion: summer wedding · Budget: $200 · Avoid: heels*). Then **Build my looks**.
5. **Results** (new, inside the Fitting Room). A live agent progress list (*Reading your style ✓ · Building looks ✓ · Checking budget…*), then three look cards. Each card has the try-on image, product thumbnails with prices and the total, the stylist note, a *"checked against your brief"* badge, **Save**, **Show another**, and a small mic for **Refine by voice**.
6. **My Looks** (Project 1, now backed by Firestore). Saved looks with their try-on image, date and the request that created them. The empty state *"Your closet is waiting."* stays.
7. **Shop** (Project 1). The same grid and filters, now loading the real-photo catalog from `GET /api/products`.

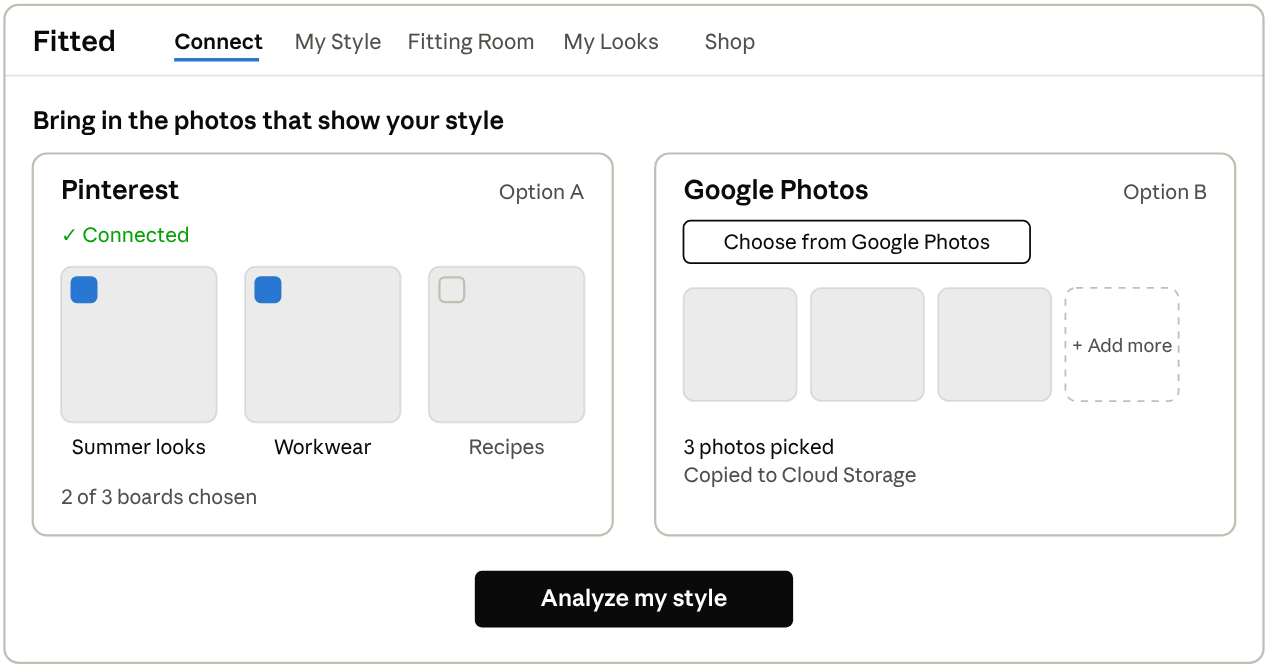
Fitting Room: speak a request, get three checked looks with a try-on



Wireframe · Fitting Room with voice request and Results (screens 4–5)

The photo and voice request sit on the left. The parsed brief, live agent progress and three look cards fill the right, and every card ties back to both the pins and the spoken request.

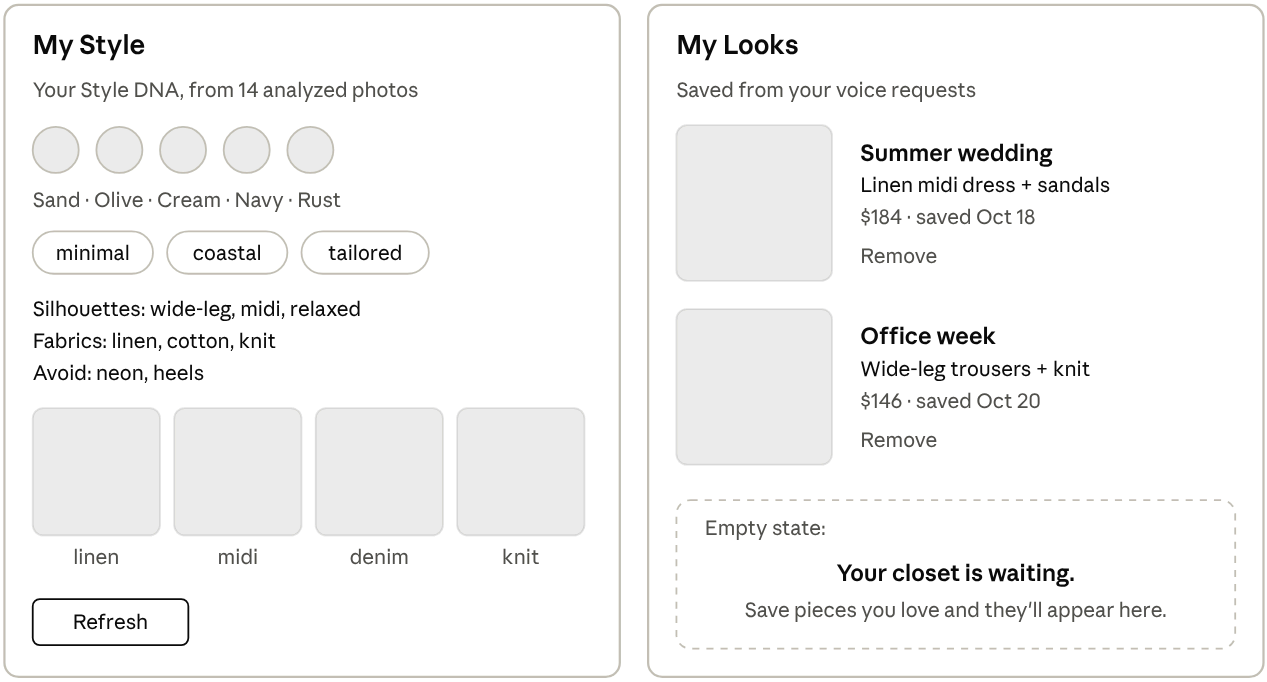
Connect: choose Pinterest boards or Google Photos; both feed the same Style DNA



Wireframe · Connect screen with both photo options (screen 2)

Both options end in the same **Analyze my style** step, so everything after Connect works the same whichever source the user picks.

My Style shows what Gemini learned; My Looks keeps what the user saved



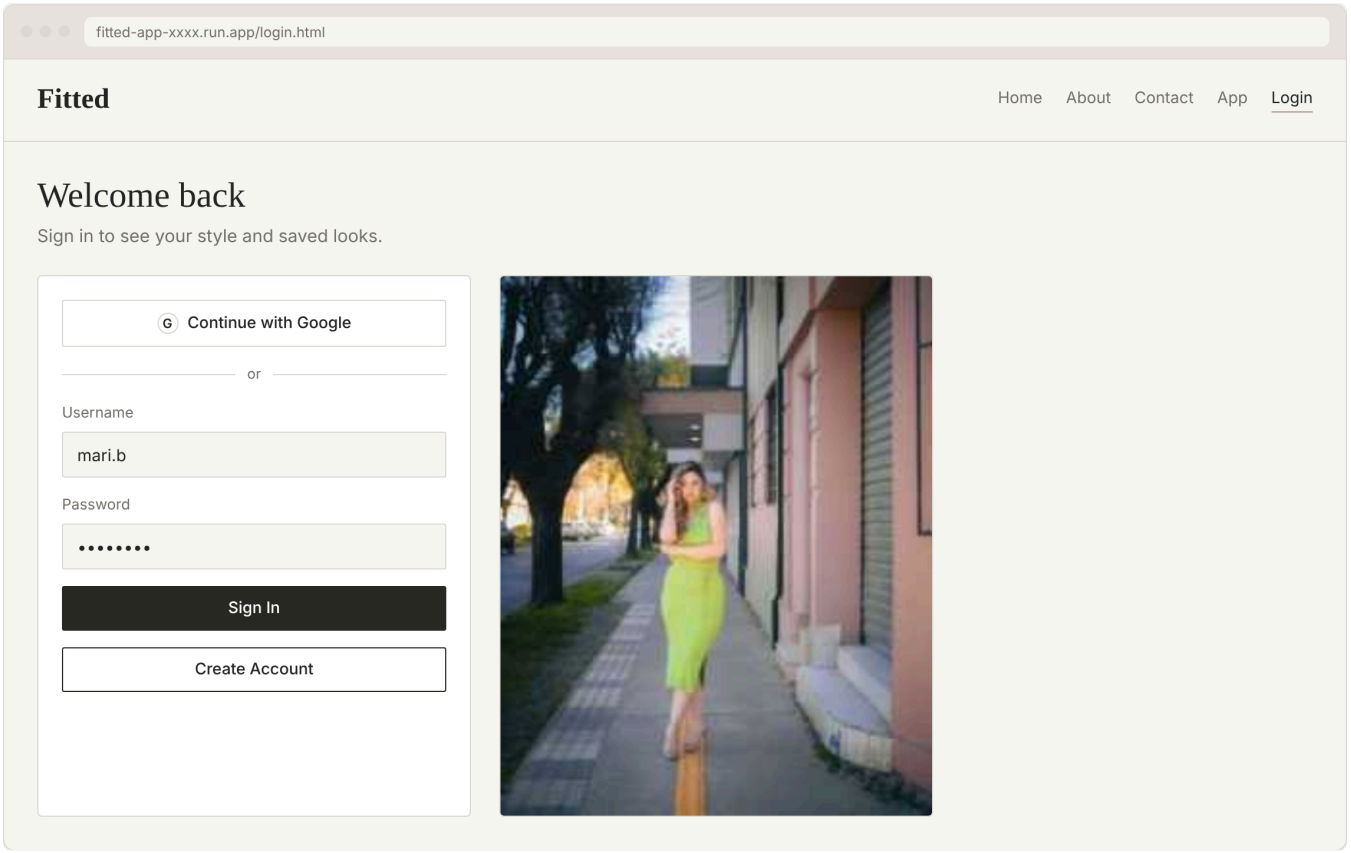
Wireframe · My Style and My Looks (screens 3 and 6)

My Style is read from `users/{uid}.styleDna` and `photoAnalyses`; My Looks is the Firestore query for saved looks, and **Remove** deletes the document.

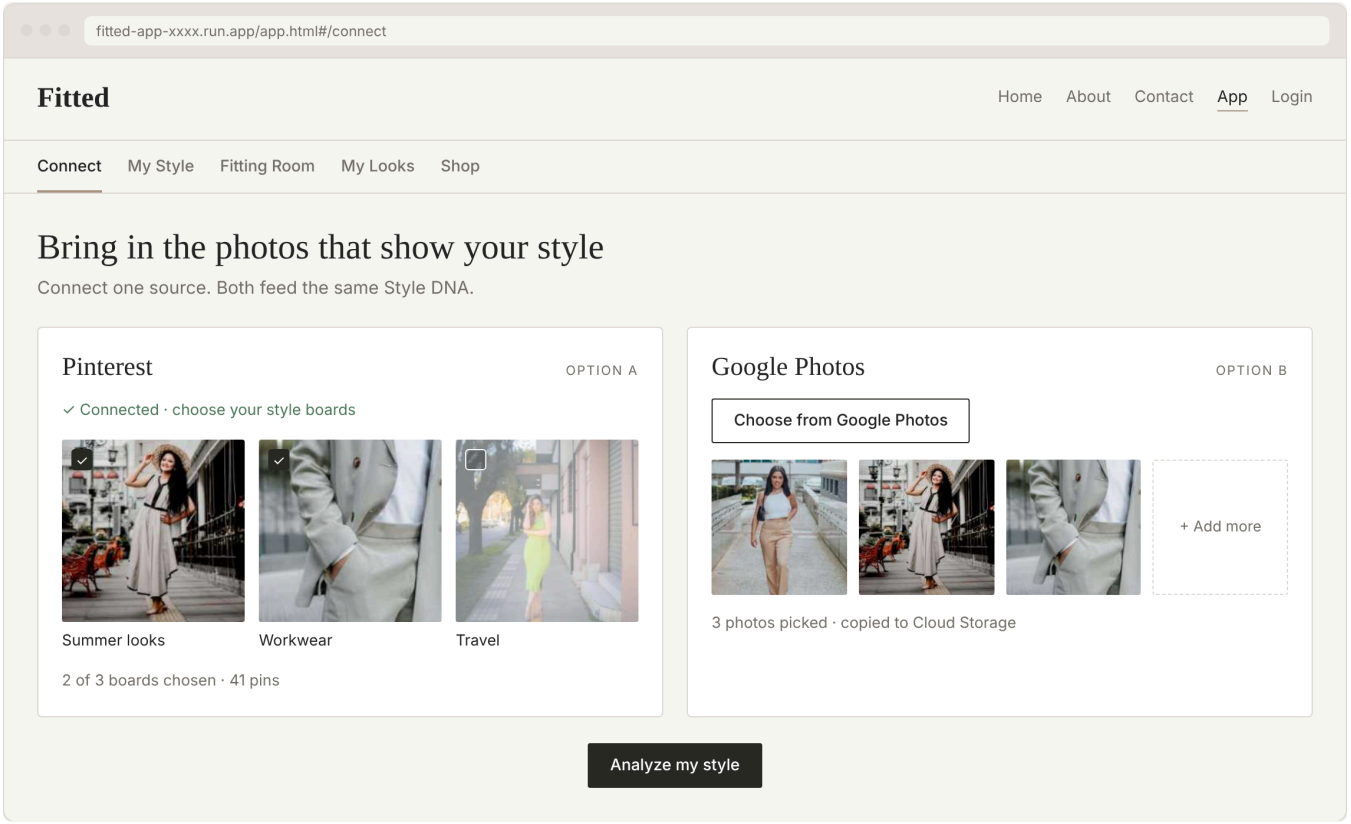
Website mockups

The same screens built as a clickable web page in Fitted's Project 1 style: open the Fitted website mockups. The Fitting Room's mic button replays the voice request and the agents building looks. Photos of people wearing the outfits are free stock photos from Pexels; product thumbnails are Project 1's placeholder drawings. In the built app, try-on images come from Gemini.

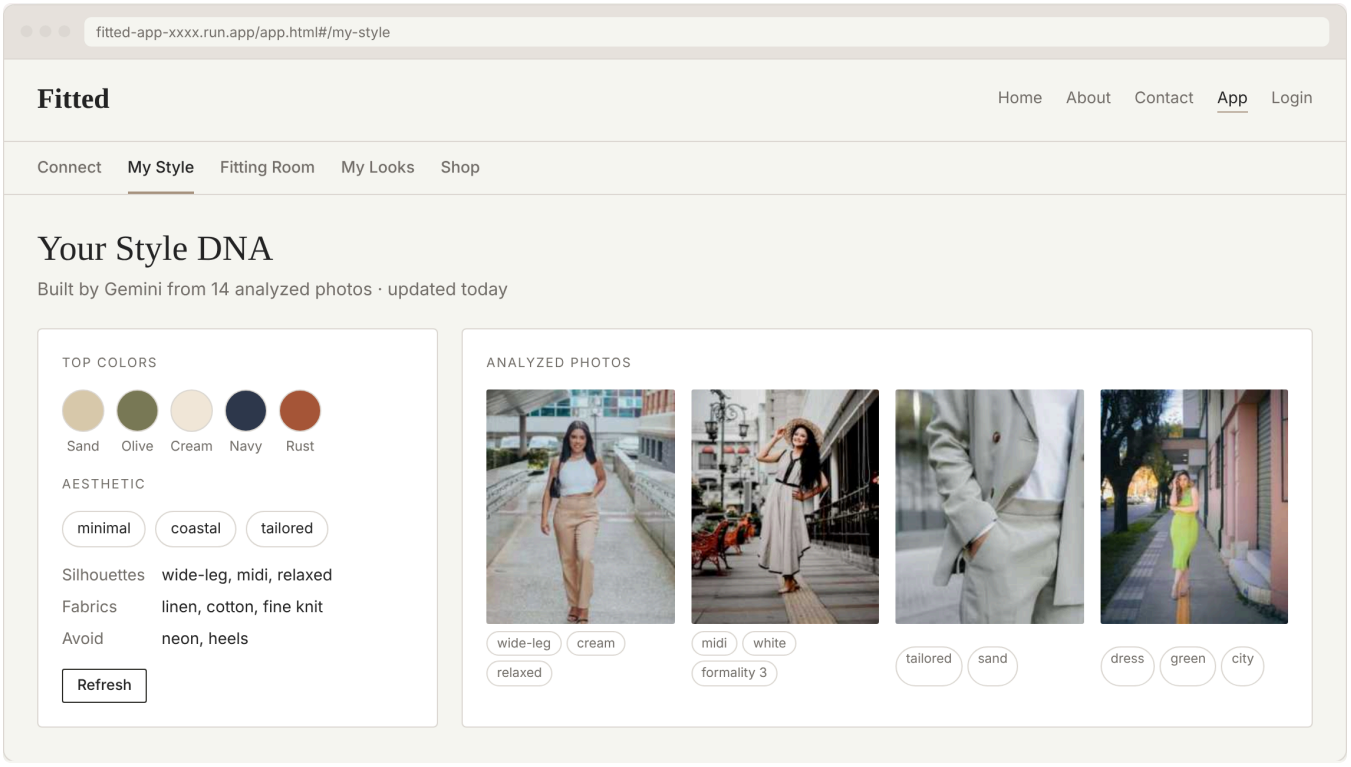
1 · Login



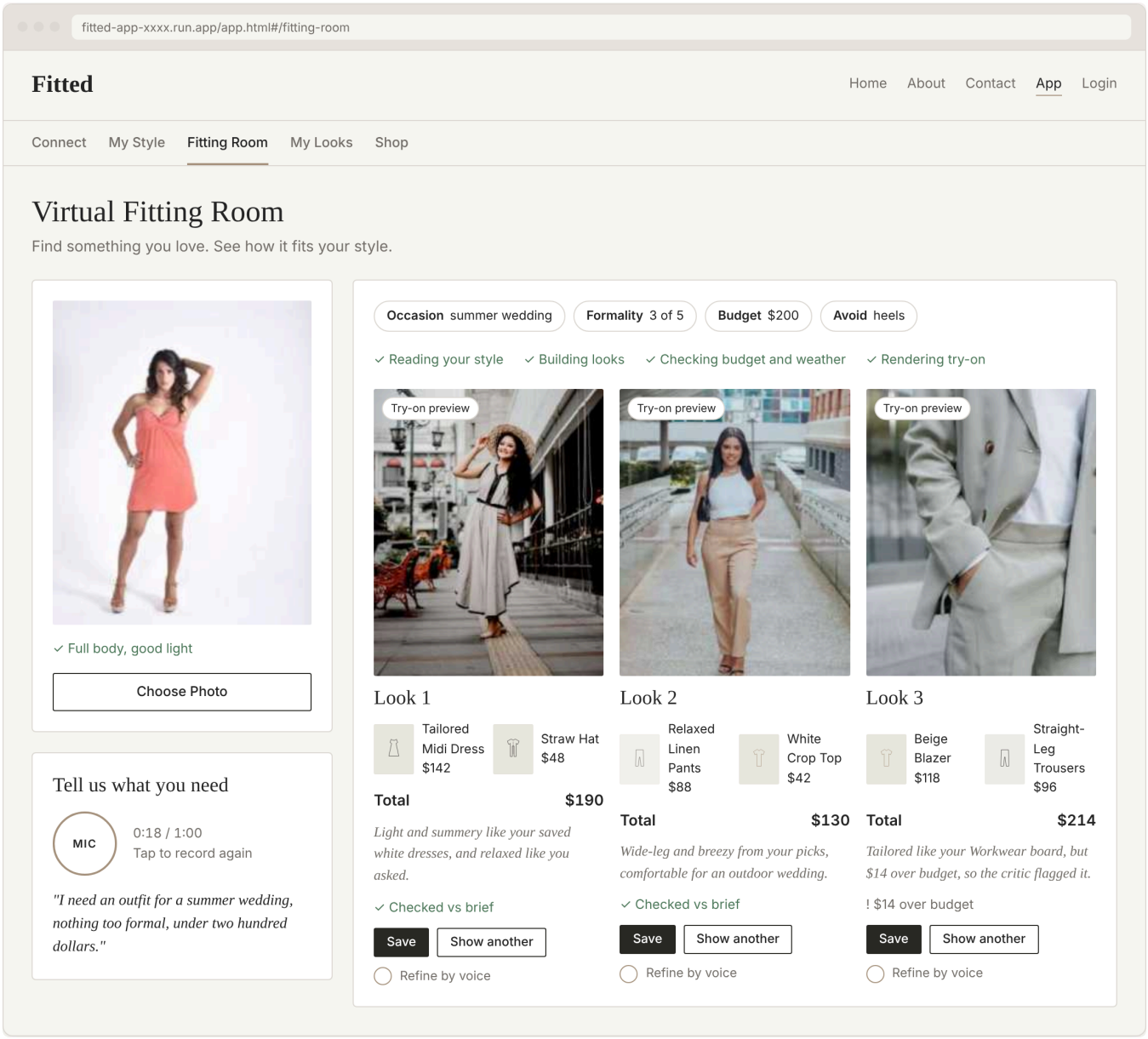
2 · Connect (Option A Pinterest, Option B Google Photos)



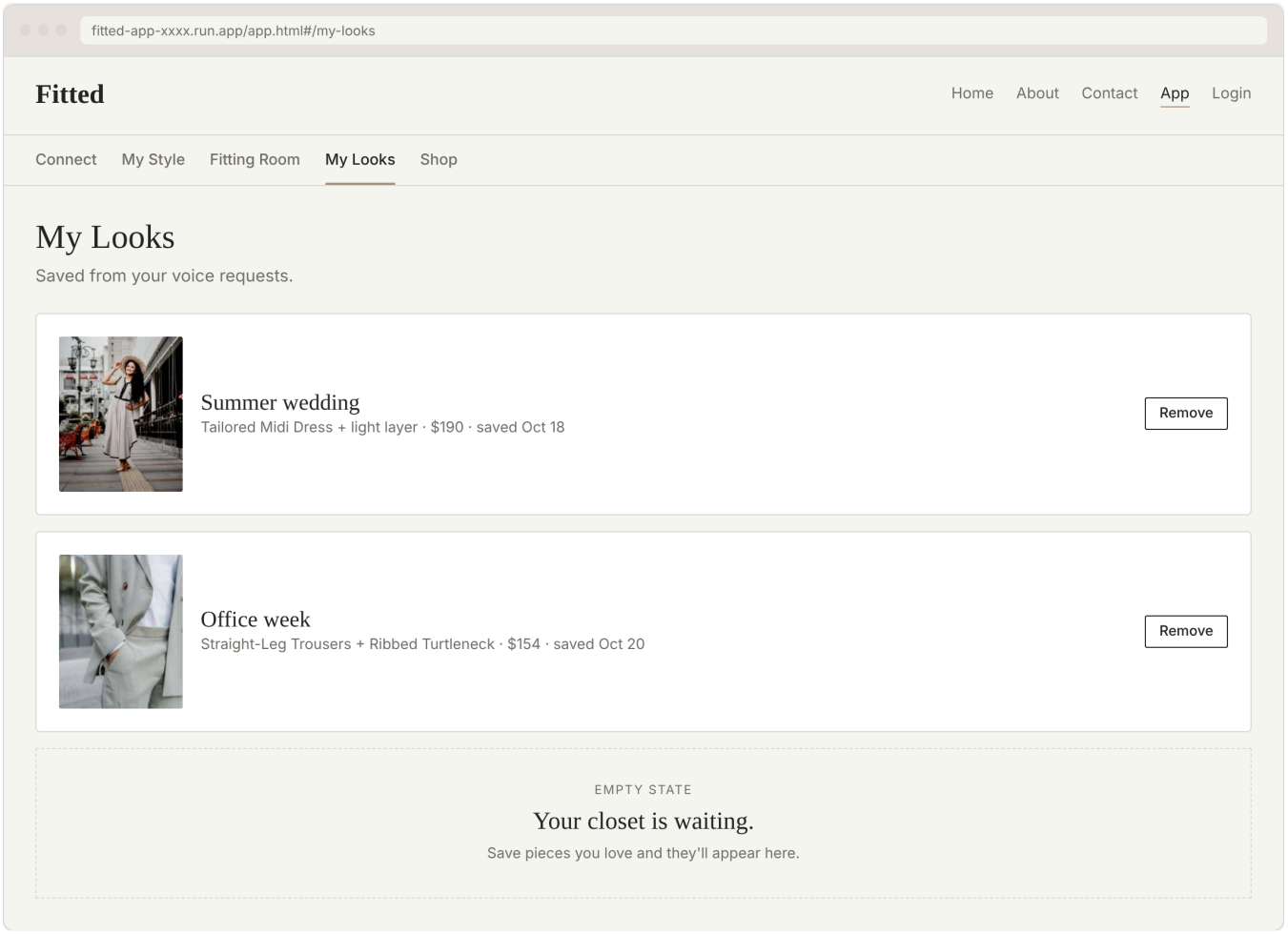
3 · My Style



4–5 · Fitting Room with Results


fitted-app-xxxx.run.app/app.html#/fitting-room
Fitted
Home
About
Contact
App
Login
Connect
My Style
Fitting Room
My Looks
Shop
Virtual Fitting Room
Find something you love. See how it fits your style.
✓ Full body, good light
Choose Photo
Tell us what you need
MIC
0:18 / 1:00
Tap to record again
"I need an outfit for a summer wedding, nothing too formal, under two hundred dollars."
Occasion summer wedding
Formality 3 of 5
Budget $200
Avoid heels
✓ Reading your style
✓ Building looks
✓ Checking budget and weather
✓ Rendering try-on
Try-on preview
Look 1
Tailored Midi Dress $142
Straw Hat $48
Total $190
Light and summery like your saved white dresses, and relaxed like you asked.
✓ Checked vs brief
Save
Show another
Refine by voice
Try-on preview
Look 2
Relaxed Linen Pants $88
White Crop Top $42
Total $130
Wide-leg and breezy from your picks, comfortable for an outdoor wedding.
✓ Checked vs brief
Save
Show another
Refine by voice
Try-on preview
Look 3
Beige Blazer $118
Straight-Leg Trousers $96
Total $214
Tailored like your Workwear board, but $14 over budget, so the critic flagged it.
! $14 over budget
Save
Show another
Refine by voice

6 · My Looks


fitted-app-xxxx.run.app/app.html#/my-looks
Fitted
Home
About
Contact
App
Login
Connect
My Style
Fitting Room
My Looks
Shop
My Looks
Saved from your voice requests.
Summer wedding
Tailored Midi Dress + light layer · $190 · saved Oct 18
Remove
Office week
Straight-Leg Trousers + Ribbed Turtleneck · $154 · saved Oct 20
Remove
EMPTY STATE
Your closet is waiting.
Save pieces you love and they'll appear here.

7 · Shop

fitted-app-xxxx.run.app/app.html#/shop

Fitted

Home About Contact App Login

Connect My Style Fitting Room My Looks Shop

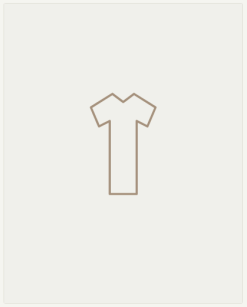
Shop

The catalog the agents choose from, loaded from Firestore.

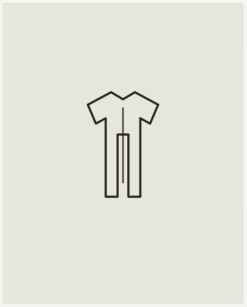
All Tops Outerwear Dresses Bottoms



TOPS
Essential Cotton Tee $38



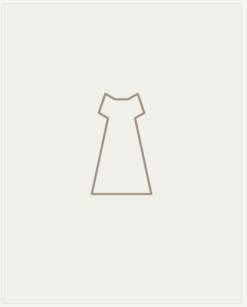
TOPS
Ribbed Turtleneck $58



OUTERWEAR
Wool-Blend Overcoat $248



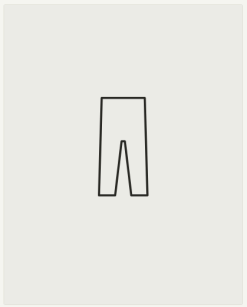
OUTERWEAR
Cropped Denim Jacket $132



DRESSES
Silk Slip Dress $168



DRESSES
Tailored Midi Dress $142



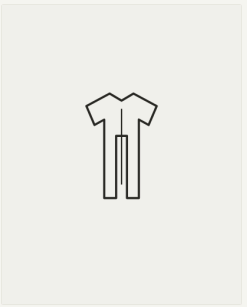
BOTTOMS
Straight-Leg Trousers $96



BOTTOMS
Relaxed Linen Pants $88

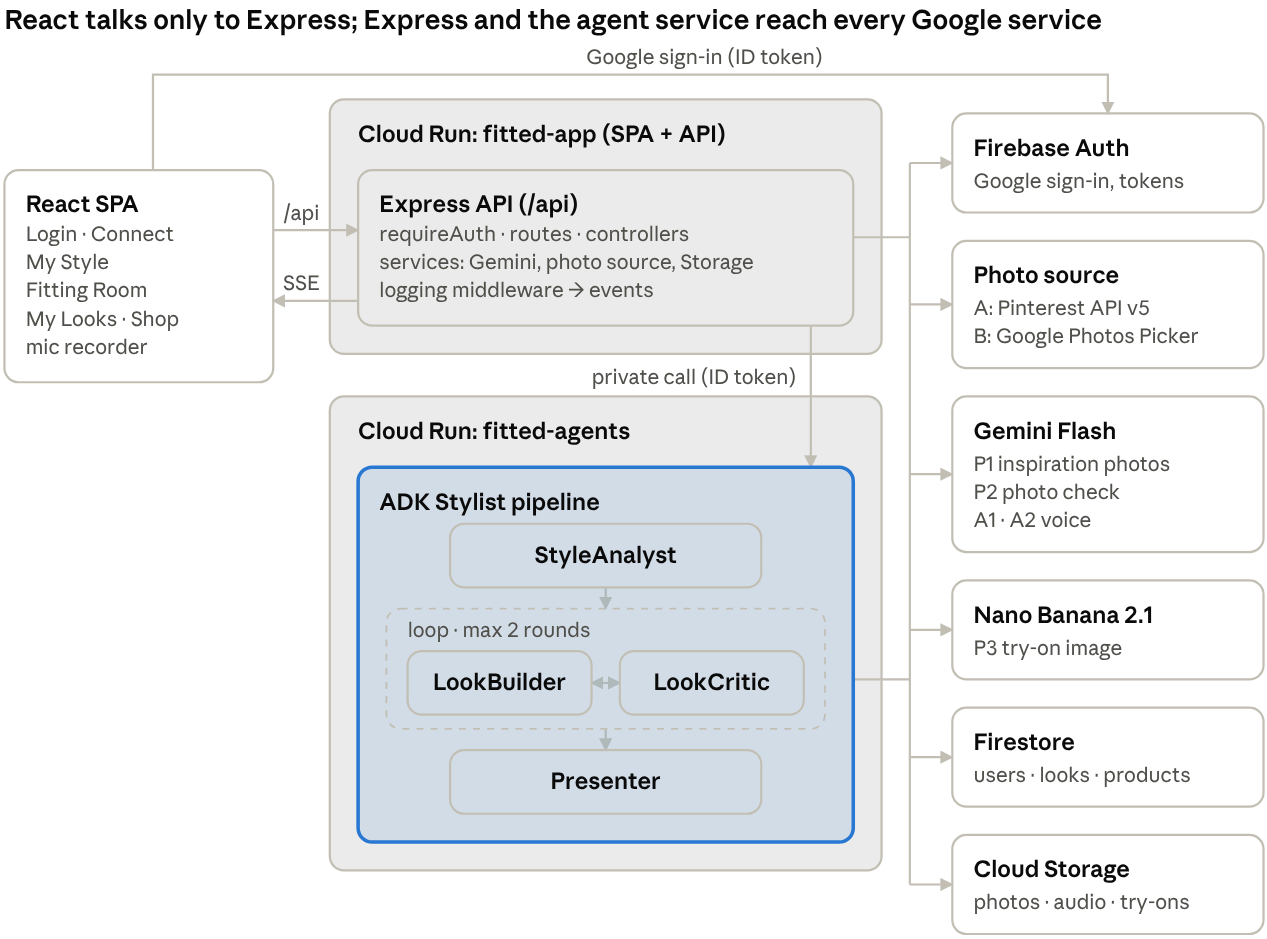


TOPS
Oversized Button-Down $74



OUTERWEAR
Quilted Field Jacket $188

System diagram



Secrets (Gemini key, OAuth client secrets) come from Secret Manager; every request is logged to Firestore events.

System diagram · React SPA, Cloud Run (Express + ADK agents), 6 Google and Pinterest services

The browser only calls `/api` on the `fitted-app` Cloud Run service and receives agent progress back over Server-Sent Events. Express calls the private `fitted-agents` service, where the four ADK agents run. Both services share the same clients for the photo source (Pinterest or Google Photos), Gemini, Firestore and Cloud Storage. Firebase Auth is the only service the browser talks to directly, to sign in.

Flow of control

The browser only talks to our Express API, and Express is the only part that talks to Pinterest, Gemini, the ADK agents, Firestore and Cloud Storage. Each step below lists the route and the technology it uses.

1. **Launch.** The **App** link on our Project 1 site opens the SPA on the `fitted-app` Cloud Run service. One container serves the built React files with `express.static` and the API under `/api`.
2. **Sign in.** React calls Firebase Auth `signInWithPopup(GoogleAuthProvider)` and gets an ID token. Every `/api` request carries it. The `requireAuth` middleware calls `admin.auth().verifyIdToken()` and attaches `req.uid`. The first visit creates `users/{uid}`.
3. **Connect a photo source.**
   - *Option A, Pinterest:* the OAuth flow under Authentication, then `GET /api/photos/pinterest/boards` calls Pinterest `GET /v5/boards`, and `PUT /api/me/boards` saves the ticked board IDs. Pins are listed with `GET /v5/boards/{id}/pins`, following the `bookmark` cursor.
   - *Option B, Google Photos:* `POST /api/photos/google/session` creates a Picker session and returns its `pickerUri`. React opens it, and `GET /api/photos/google/session/:id` polls until `mediaItemsSet` is true, then lists the picked items.
   - Either way, `photoSourceService` returns one list of new images, which are copied to Cloud Storage and resized with `sharp`.
4. **Analyze style.** `POST /api/style/analyze` skips images already in `photoAnalyses`, sends new ones to Gemini **P1** in batches with a JSON schema, and writes each result. Express then asks `fitted-agents` to run **StyleAnalyst**, which writes `styleDna` to `users/{uid}`.
5. **Upload photo.** `POST /api/photo` (multipart, `multer`) stores the image in Cloud Storage and runs Gemini **P2**. The check result is shown under the photo.
6. **Record a request.** React uses `getUserMedia` and `MediaRecorder` with a 60-second limit. It picks the first format the browser supports with `MediaRecorder.isTypeSupported()`: `audio/webm;codecs=opus` (Chrome, Edge, Firefox), otherwise `audio/mp4` (Safari). `POST /api/briefs` sends the recording and its MIME type to Gemini **A1** inline. The server saves the audio to Cloud Storage and the brief to `briefs`. Edits to the brief chips are saved with `PATCH /api/briefs/:id`.
7. **Build looks.** `POST /api/looks/build { briefId }` makes an authenticated call to `fitted-agents`, which runs the Stylist pipeline with session state `{ uid, briefId }`. Express relays the agents' progress to React over Server-Sent Events (`GET /api/looks/stream/:runId`):
   a. **StyleAnalyst** reads the Style DNA.

   b. **LookBuilder** and **LookCritic** run their loop, calling `searchCatalog` on Firestore `products`.
   c. **Presenter** calls Gemini **P3** (Nano Banana) with the user's photo and the real product photos, stores the try-on in Cloud Storage, and writes 3 documents to `looks`.
   d. The final event sends the looks, with signed image URLs, to the **Results** screen.
8. **Refine by voice.** `POST /api/looks/refine` sends the new recording to Gemini **A2**, which returns changes to the brief. LookBuilder → LookCritic → Presenter run again in the same ADK session.
9. **Save and manage.** `PATCH /api/looks/:id` sets `saved` or a thumbs up/down, and `DELETE /api/looks/:id` removes a look. **My Looks** loads `GET /api/looks?saved=true`, and thumbs feedback feeds the next StyleAnalyst run.
10. **Log for analytics.** Server middleware writes every request to `events` (route, model, latency, tokens, status), and the React app sends user actions to Google Analytics 4 (see Analytics plan).

Deployment: two Cloud Run services built from the same repository. `fitted-app` uses a multi-stage Dockerfile, like Project 1's, that runs `vite build` for `frontend/` and copies the result into the Node image running `backend/app.js`. `fitted-agents` is built from `backend/agents/` and deployed with ingress limited to authenticated callers. Both are deployed with `gcloud run deploy`. Secrets (Gemini key, Pinterest and Google OAuth client secrets) come from Secret Manager, and each service account has only the Firestore and Cloud Storage roles it needs.

How the Project 1 React app evolves

The Project 1 SPA moves into `frontend/` almost unchanged. Its components are reused, and Project 2 adds new screens around them and replaces the mock data and the simulated try-on with real API calls.

Reused from Project 1:

Project 1 piece	Project 2 change
`Navbar`, `SpaTabs`	Same components. SpaTabs gains **Connect** and **My Style** tabs.

Project 1 piece	Project 2 change
`ProductCard`, `ProductGrid`, `CategoryFilter`, `FavoritesButton`	Unchanged. They now display Firestore products and the pieces inside each look.
`PhotoUploader`	Uploads to `POST /api/photo` instead of preview-only, and shows the Gemini photo check.
`SelectedProduct`, `TryOnPanel`	TryOnPanel shows the real Nano Banana try-on image instead of the *"will be connected"* message.
`FittingRoomPage`, `ShopPage`, `MyLooksPage`	Same routes. Fitting Room hosts the voice and results flow; My Looks reads saved looks from Firestore.
`LoginForm`, `RegisterForm`, `LoginApp`	Same layout and state logic. Sign-in goes through Firebase Google Sign-In.
`services/tryOnService.js`	`simulateTryOn()` becomes `buildLooks()` and calls Express. This is the swap Project 1 prepared for.
`data/products.js`	Becomes a one-time seed script for the Firestore `products` collection.

New React pieces:

- **Components:** `GoogleSignInButton`, `PhotoSourceCard`, `BoardPicker`, `StyleDnaCard`, `ColorSwatch`, `VoiceRecorder`, `BriefChips`, `AgentProgress`, `LookCard`, `LookResults`.
- **Pages:** `ConnectPage`, `MyStylePage`.
- **Hooks:** `useAuth` (Firebase user and token), `useVoiceRecorder` (MediaRecorder state and timer), `useLookStream` (Server-Sent Events from the agents).
- **Services:** `apiClient.js`, one `fetch` wrapper that adds the ID token to every request.

How the backend connects: the React code only ever calls `/api/*` through `apiClient.js`. Because Express serves both the SPA and the API from the same Cloud Run URL, there are no cross-origin (CORS) issues. In development, Vite's proxy forwards `/api` to `localhost:8080`.

Repository layout: `frontend/` (React SPA) and `backend/` (`routes/`, `controllers/`, `middleware/`, `services/` for Gemini, Pinterest, Google Photos, Firestore and Storage, `agents/` for ADK, `app.js`), plus `README.md` and `documentation/`, matching the structure in the assignment.

Analytics plan

We will collect usage data two ways and compare them in the Analytics + Logging report.

	Google Analytics 4 (client side)	Server logging (Cloud Logging + Firestore `events`)
What it records	Page and tab views, plus custom events: `photo_source_connected`, `style_analyzed`, `voice_recorded`, `looks_built`, `look_saved`, `refine_by_voice`	Every `/api` request: route, status, latency, Gemini model, input/output tokens, agent run time, errors
Answers	How people move through the app and where they drop off	How fast and how expensive each Gemini and agent step is, and what fails
Limits	Blocked by some ad blockers; no backend timing	No view of what users click between requests

The report will compare the two on the same test sessions: counts of looks built and saved, the funnel from Connect to Save, Gemini latency and token cost per look, and where the numbers disagree.

Testing plan

- **Browsers:** Chrome, Safari and Edge on desktop, plus Safari on iPhone and Chrome on Android. Internet Explorer has been retired by Microsoft, so Edge replaces it. Safari is the key test for voice recording (MP4 format).
- **Users:** at least two team members' own accounts, each with their own photos and voice requests, as the wiki's demonstration page requires.
- **Change tests:** add a new pin or picked photo and show the Style DNA and looks change; record a different request and show different looks.

Submission

This proposal is exported as a PDF and linked on the **proposal** page of our Project 2 GitHub wiki. The GitHub repository URL is posted to Canvas → Discussion Board → Project 2 Proposals.

Sources

- Pinterest — Understanding our access tiers
- Pinterest — Set up your app
- Pinterest Business Community — Understanding the PINNER_DATA_ACCESS_DENIED error
- Pinterest API Guide 2026: v5 access, OAuth, scopes and limits (upload-post.com)
- Gemini API — Audio understanding
- Gemini API — Image generation (Nano Banana)
- Agent Development Kit (ADK)
- Google Photos Picker API — Get started